Data Mining and Machine Learning

Gennaro Basile

March 25, 2026

Contents

1 Gradient Descent and Backpropagation **3**

1.1 The Geometry of Optimization . . . 3

1.1.1 The Reality of the Loss Landscape. . . . 3

1.1.2 Different paths toward the minimum. . . . 4

1.1.3 Visualizing the Loss Landscape: Filter Normalization . . . 5

1.2 The Mathematics of Gradient Descent . . . 5

1.2.1 When to Stop Training (Convergence vs. Overfitting) . . . 7

1.3 Stochastic Gradient Descent (SGD) . . . 8

1.3.1 Terminology of SGD . . . 8

1.4 Backpropation Method . . . 8

1.4.1 The Computational Graph . . . 9

1.4.2 Standard Mathematical Notation . . . 9

1.4.3 The Two Assumptions of the Cost (Loss) Function . . . 9

1.5 The Four Fundamental Equations of Backpropagation . . . 10

1.5.1 The Concept of "Neuron Error" (δ) . . . 10

1.5.2 BP1: Error in the Output Layer . . . 10

1.5.3 BP2: Propagating the Error Backward (Hidden Layers) . . . 10

1.5.4 BP3: Derivative of the Cost with Respect to Biases . . . 11

1.5.5 BP4: Derivative of the Cost with Respect to Weights . . . 11

1.5.6 The Complete Algorithm Summary . . . 11

1.6 Implications of Backpropagation: Learning Dynamics and Bottlenecks . . . 11

1.6.1 Learning Speed and Gradient Magnitude . . . 11

1.6.2 The Saturation Problem . . . 12

1.6.3 The Vanishing Gradient Problem in Deep Networks . . . 12

1.6.4 The Complete Training Loop (Stochastic Gradient Descent) . . . 12

1 Gradient Descent and Backpropagation

1.1 The Geometry of Optimization

Once a neural network makes a prediction and calculates the error (Loss/Cost), it must adjust its weights to improve. The algorithm used to find the optimal weights is called **Gradient Descent**.

Gradient Descent

Gradient Descent is an iterative optimization method used to minimize a function by moving the parameters step by step in the direction opposite to the gradient.

To understand Gradient Descent, we must visualize the problem geometrically. Imagine the **Cost Function** as a physical landscape (a hypersurface) where the coordinates are the model's weights, and the elevation is the error.

- **High elevation:** High error (bad predictions).
- **Low elevation (Valleys):** Low error (good predictions).

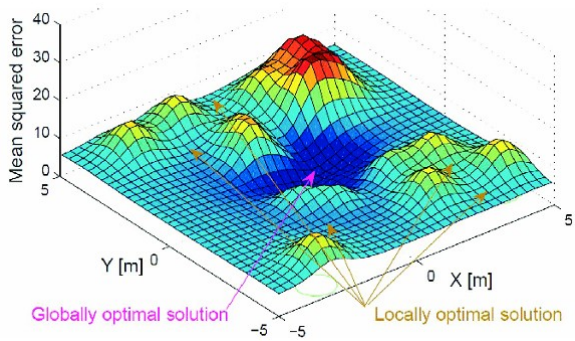

Figure 1.1.1: *The Cost Hypersuperface*

Cost Hypersurface

A cost hypersurface is the geometric representation of the cost function in the parameter space of the model. Each point of this space corresponds to one specific configuration of the model parameters.

The Goal of Training: To navigate the network's weights down the landscape until they reach the **Global Minimum** (the absolute lowest possible error).

Global Minimum

The global minimum is the absolute lowest point of the cost function over the entire parameter space.

Local Minimum

A local minimum is a point whose cost is lower than that of nearby points, but not necessarily the lowest over the entire parameter space.

1.1.1 The Reality of the Loss Landscape.

In simple models (like Linear Regression), this landscape is a smooth, bowl-shaped **Convex Function** with only one perfect bottom.

Convex Function

A convex function is a function whose surface has only one global minimum and no local minima distinct from it.

However, in Deep Learning (with millions of parameters), the landscape is highly **Non-Convex**. It is filled with jagged peaks, flat plateaus, and **Local Minima** (shallow valleys that trap the model before it reaches the true bottom).

Non-Convex Function

A non-convex function is a function whose surface may contain many local minima, local maxima, saddle regions, and irregular shapes.

1.1.2 Different paths toward the minimum.

The path followed during optimization depends on many factors. The goal is to find the weight configuration that minimizes the cost function, but convergence does not always happen in the same way.The speed of convergence depends on:

- the starting point,
- the shape of the cost surface,
- the step size used during exploration.

The figure (1.1.2) shows two different behaviors. In one case, the path is well-behaved and quickly reaches the minimum. In the other case, the path oscillates inside a narrow valley and converges more slowly.

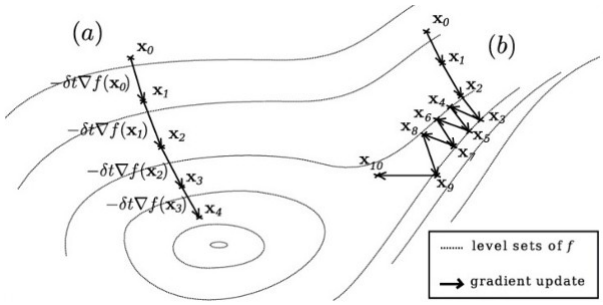

Figure 1.1.2: *Different path for the minimum*

This tells us that optimization is not only about the formula, but also about the geometry of the problem. A bad learning rate or a difficult landscape can make training unstable or slow.

Convergence

Convergence is the process through which an iterative optimization algorithm approaches a stable solution, ideally a minimum of the cost function.

In practice, convergence means that after several updates the parameters are no longer changing significantly, and the cost function does not improve much anymore.

Researchers use advanced projection techniques, like *Filter Normalization*, to properly visualize these massive, chaotic multidimensional landscapes in 3D to understand how models learn.

1.1.3 Visualizing the Loss Landscape: Filter Normalization

Neural network loss landscapes exist in millions of dimensions. To visualize them, we plot a 2D slice as a 3D mountain range (where height = error). However, doing this naively creates a problem.

- **The Problem (Scale Invariance):** In modern networks, you can scale weights up and down mathematically without changing the final prediction. For example:
  - we multiply the weights of one layer by a constant,
  - and divide the weights of the following layer by the same constant.

  Plotting these raw, unadjusted weights creates optical illusions, making smooth, easily trained valleys look like chaotic, jagged cliffs.
- **The Solution (Filter Normalization):** To reveal the *true* geometry of the landscape, we must scale our X and Y graphing axes to perfectly match the actual size (norm) of the model's filters.

The 5-Step Visualization Process

1. **Center:** Start at the fully trained model's parameter vector (θ^*).
2. **Axes:** Pick two random direction vectors (δ and η) to act as your X and Y axes.
3. **Normalize:** Rescale δ and η so they match the scale of the model's filters.
4. **Grid Evaluation:** Shift slightly around the center point and calculate the Loss across a 2D grid using the formula:
$$\theta = \theta^* + x\delta + y\eta$$
5. **Plot:** Map these Loss values to the Z-axis (height) to create an accurate 3D surface map.

Filter Normalization

Filter normalization is a visualization technique in which the directions used to explore the loss landscape are rescaled according to the norm of the model filters, so that the plotted surface is not distorted by scale invariance.

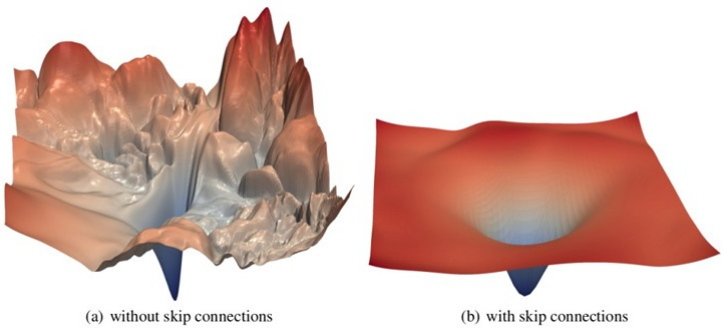

Figure 1.1.3: *Filter Normalization*

In short, filter normalization helps us visualize the true shape of the loss landscape around the final trained model, avoiding misleading distortions caused by the scaling of the weights.

1.2 The Mathematics of Gradient Descent

We defined Gradient Descent as a numerical method used to find the minimum of a function of many variables. In this context, the function is the cost function C, and the variables are the weights of the model.

Cost Function

The cost function is a function that measures how far the predictions of a model are from the desired targets over the training data.

The cost is computed over the entire training dataset at each iteration. We can write the approximate variation of the cost:

$$\Delta C \sim \sum_{i=1}^{n} \frac{\partial C}{\partial w_i} \Delta w_i,$$

where n is the number of weights.

This formula is the first-order approximation of how the cost changes when the weights are perturbed. It is a multivariable version of the linear approximation from calculus.

Partial Derivative

A partial derivative measures how a multivariable function changes when only one variable is varied while all the others are kept fixed.

In optimization, the partial derivative $\frac{\partial C}{\partial w_i}$ tells us how sensitive the cost is to changes in the weight w_i.

Link between ΔC and Δw. We can rewrites the previous expression using vector notation and introduces the gradient.

Gradient

The gradient of a function is the vector containing all its partial derivatives. It points in the direction of the greatest increase of the function.

It is possible states that

$$\Delta C \sim \nabla C \cdot \Delta w,$$

where

$$\nabla C = \left(\frac{\partial C}{\partial w_1}, \ldots, \frac{\partial C}{\partial w_n} \right)^T$$

is the gradient column vector, and Δw is the vector of weight variations.

The dot product $\nabla C \cdot \Delta w$ tells us how the cost changes when we move in the direction Δw. If we want the cost to decrease, we should choose Δw so that this dot product becomes negative. we introduce a small positive parameter η and pose that:

$$\Delta w = -\eta \nabla C,$$

where $\eta > 0$ is a small parameter. Substituting this into the previous formula gives:

$$\Delta C \sim -\eta \nabla C \cdot \nabla C = -\eta \|\nabla C\|^2. \tag{1}$$

Since $\|\nabla C\|^2 \geq 0$ and $\eta > 0$, we obtain $\Delta C < 0$ whenever the gradient is nonzero. This guarantees that, locally, the cost decreases.

Dot Product

The dot product between two vectors measures how much one vector points in the direction of the other.

Learning Rate

The learning rate is a positive scalar that controls the size of the update step during optimization.

This is the central mathematical intuition of Gradient Descent: move in the opposite direction of the gradient because that is the direction of the steepest local decrease.

This gives us the fundamental **Weight Update Rule** of Machine Learning:

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla C \tag{2}$$

My new weight equals my old weight, minus a small step in the direction that reduces the error.

The Danger of the Learning Rate (η). The Learning Rate is arguably the most important hyperparameter in Deep Learning. It dictates how large of a step the algorithm takes down the mountain and on the geometry of the cost function.

- **If η is too small:** convergence may be very slow.
- **If η is too large:** the algorithm may overshoot the minimum and oscillate.

So the elegance of Gradient Descent lies in its simplicity, while its practical difficulty lies in choosing appropriate hyperparameters and handling difficult optimization landscapes.

1.2.1 When to Stop Training (Convergence vs. Overfitting)

Training should not continue indefinitely. We monitor the Cost (Loss) Function during training to decide when to stop:

1. **Convergence:** Stop when the loss function reaches a flat plateau and stops decreasing. The model has learned all it can.
2. **Early Stopping (Preventing Overfitting):** We must monitor the loss on both the *Training Set* and the *Validation Set*. If the Training Loss continues to drop, but the Validation Loss starts to *increase*, the model is **Overfitting** (memorizing the training data instead of learning general patterns). Training must be stopped immediately.

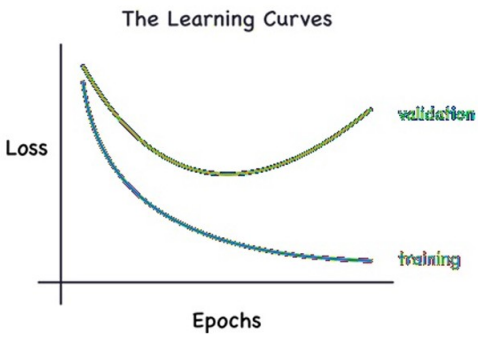

Figure 1.2.1: *Loss Validation vs Training set*

In traditional data science, having too many dimensions (features) is bad: it causes data sparsity and increases compute time (The Curse of Dimensionality).

However, in Deep Learning optimization, high dimensionality is actually a blessing. In a 3D landscape, it is easy to get stuck in a valley (Local Minimum) because there are only two directions to move (X and Y). In a 1-million-dimensional landscape, if 999,999 directions go

uphill, there is almost certainly at least one direction that still goes downhill, allowing the algorithm to escape the trap and continue learning.

1.3 Stochastic Gradient Descent (SGD)

Standard Gradient Descent has a massive computational flaw: to take a *single* step down the mountain, it must calculate the error across the *entire* training dataset. If your dataset has 10 million images, computing 10 million errors before updating the weights once is too slow.

The solution is **Stochastic Gradient Descent (SGD)** using **Mini-Batches**. Instead of looking at the whole dataset, SGD grabs a small, random handful of data (a mini-batch, e.g., 32 or 64 images). It calculates the gradient on just that small batch, and updates the weights immediately.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is an optimization method in which the gradient is estimated using only a small randomly selected subset of the training data instead of the entire dataset.

- **Trade-off:** The gradient calculated on a mini-batch is *noisy* and imprecise compared to the exact gradient of the whole dataset. The path down the mountain will be jagged and zig-zagged.
- **Benefit:** Because it updates the weights instantly after just 32 images, the model trains exponentially faster.

Mini-batch update algorithm, batch, and epoch The practical algorithm used in mini-batch learning can be summerized like:

- `Step 1`: select m samples to form a mini-batch.
- `Step 2`: compute the gradient approximation on that mini-batch.
- `Step 3`: update the weights and biases using

$$w'_k = w_k - \frac{\eta}{m} \sum_j \frac{\partial C_{x_j}}{\partial w_k}, \qquad b'_l = b_l - \frac{\eta}{m} \sum_j \frac{\partial C_{x_j}}{\partial b_l}.$$

- `Step 4`: choose another random mini-batch and repeat the process.
- `Step 5`: continue until the entire training set has been processed.
- `Step 6`: repeat the whole process for multiple passes until convergence is reached.

1.3.1 Terminology of SGD

- **Mini-Batch:** The small subset of data used for a single weight update.
- **Iteration (or Step):** One update of the model's weights based on one mini-batch.
- **Epoch:** One complete pass through the *entire* training dataset. (If your dataset has 1,000 images and your batch size is 100, it takes 10 iterations to complete 1 Epoch).

1.4 Backpropation Method

As established, **Backpropagation** calculates the gradient of the Loss function with respect to the network's weights and biases, allowing us to update them via Gradient Descent.

Historically, backpropagation is a specific application of **Reverse-Mode Automatic Differentiation**. Popularized in 1986 by Rumelhart, Hinton, and Williams, it is the algorithm that single-handedly made the training of deep, multi-layer neural networks computationally feasible.

1.4.1 The Computational Graph

A neural network is best understood mathematically as a **Computational Graph**: a massive composite function broken down into a sequence of smaller, trackable operations. This allows us to split the training process into two phases:

1. **Forward Phase:** The input flows left-to-right through the graph to compute the final prediction and the Loss.
2. **Backward Phase:** The error flows right-to-left, using the Chain Rule to calculate gradients for every parameter.

Computational Graph

A **computational graph** is a graph representation of a computation, where nodes correspond to operations or variables, and edges represent the flow of values between them.

1.4.2 Standard Mathematical Notation

To perform backpropagation on a large scale, we must use precise notation. Let l denote the current layer, and $l-1$ the previous layer.

The calculation for a layer is split into two distinct variables: the linear **Weighted Input** (z) and the non-linear **Activation** (a).

1. The Weighted Input (z). Before applying the activation function, we calculate the raw linear combination of the previous layer's outputs. For a single neuron j in layer l:

$$z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l \tag{3}$$

In compact vector/matrix notation for the entire layer:

$$\mathbf{z}^l = (\mathbf{W}^l)^T \mathbf{a}^{l-1} + \mathbf{b}^l \tag{4}$$

2. The Activation (a) We then apply the non-linear activation function (σ) element-wise to the weighted input:

$$\mathbf{a}^l = \sigma(\mathbf{z}^l) \tag{5}$$

Why separate them? During the backward phase, we will need to calculate the derivative of the activation function with respect to its input. Explicitly defining $\mathbf{z}^l$ makes calculating $\frac{\partial \mathbf{a}}{\partial \mathbf{z}}$ mathematically cleaner.

Activation of a Neuron

The **activation** of a neuron is the output value produced by applying the activation function to the weighted sum of the inputs plus the bias.

1.4.3 The Two Assumptions of the Cost (Loss) Function

For the backpropagation algorithm to work, the chosen Cost Function (C) must strictly satisfy two mathematical assumptions:

1. **It must be an average of individual examples**
   The total cost must be the average of the costs calculated for individual training examples (x):

$$C = \frac{1}{n} \sum_x C_x$$

Why? This allows the algorithm to calculate the gradient for a single training example (C_x) first, and then average the gradients across a batch of data.

2. **It must be a function of the network's output**
   The cost must be calculable using only the target labels and the final activation vector of the output layer (a^L). *Why?* Because backpropagation applies the Chain Rule starting from the very end of the network. If the cost function did not explicitly depend on the final output a^L, we could not calculate the first required derivative ($\frac{\partial C}{\partial a^L}$).

Example 1.1. The Quadratic Cost Function (MSE)

The classic Mean Squared Error (MSE) function perfectly satisfies both assumptions.

$$C = \frac{1}{2n} \sum_x \left\| y(x) - a^L(x) \right\|^2 \tag{6}$$

- It satisfies Assumption 1 because the $\sum_x$ averages the errors over n examples.
- It satisfies Assumption 2 because the error is directly calculated using $a^L(x)$, the network's final output.

1.5 The Four Fundamental Equations of Backpropagation

1.5.1 The Concept of "Neuron Error" (δ)

To compute the gradients efficiently without doing redundant calculus for every single parameter, backpropagation introduces an intermediate variable called the **Neuron Error**, denoted by δ_j^l (the error of neuron j in layer l).

Mathematically, it is the partial derivative of the Cost function with respect to the neuron's *weighted input* (z):

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}$$

If this error term is large, changing the neuron's input will drastically change the final cost. If it is close to zero, the neuron is already optimized (or saturated). The entire backpropagation algorithm is simply the process of calculating this δ for the output layer, passing it backwards, and using it to update the weights and biases.

1.5.2 BP1: Error in the Output Layer

The first step is to calculate the error at the very end of the network (Layer L).

$$\delta^L = \nabla_{a^L} C \odot \sigma'(z^L)$$

What it means: The error in the output layer is the product of two things:

1. $\nabla_{a^L} C$: How fast the cost changes with respect to the final prediction (e.g., Prediction minus True Target).
2. $\sigma'(z^L)$: The slope of the activation function.

The symbol $\odot$ represents the **Hadamard Product** (element-wise multiplication).

1.5.3 BP2: Propagating the Error Backward (Hidden Layers)

Once we have the error at the output layer, we calculate the error for the hidden layer immediately before it, and so on, moving backwards.

$$\delta^l = \left((W^{l+1})^T \delta^{l+1} \right) \odot \sigma'(z^l)$$

What it means: You do not need to recalculate the error from scratch. You take the error from the *next* layer (δ^{l+1}), pull it backwards through the weights connecting them ($(W^{l+1})^T$), and multiply it by the slope of the current layer's activation function ($\sigma'(z^l)$). This is the exact mechanism that allows deep networks to train efficiently.

1.5.4 BP3: Derivative of the Cost with Respect to Biases

Once the neuron error (δ) is calculated for a layer, finding the gradient for the biases is trivial.

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

What it means: The gradient of the bias is exactly equal to the neuron's error. If a neuron has a large error, its bias gets updated by a large amount.

1.5.5 BP4: Derivative of the Cost with Respect to Weights

This is the most important equation for understanding how networks learn.

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \cdot \delta_j^l$$

What it means: The gradient for any given weight is simply:

$$\text{Gradient} = \text{Incoming Activation } (a_{\text{in}}) \times \text{Outgoing Error } (\delta_{\text{out}})$$

If the incoming signal to a weight is strong, and the resulting error of the neuron it connects to is large, the weight is heavily responsible for the mistake and will receive a massive update.

1.5.6 The Complete Algorithm Summary

To train a neural network, these four equations are executed in a strict loop:

1. **Forward Pass:** Compute all weighted inputs (z) and activations (a) until you get a prediction.
2. **Output Error (BP1):** Calculate δ^L at the final layer.
3. **Backpropagate Error (BP2):** Pass the δ backwards to compute the error for all hidden layers.
4. **Calculate Gradients (BP3 & BP4):** Use the δ terms to instantly calculate the gradients for every single bias and weight in the network.
5. **Gradient Descent:** Update the parameters and repeat.

1.6 Implications of Backpropagation: Learning Dynamics and Bottlenecks

1.6.1 Learning Speed and Gradient Magnitude

The equations of Backpropagation (specifically BP4) dictate how fast a network learns.

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \cdot \delta_j^l$$

This equation reveals a harsh truth about Gradient Descent: **The speed at which a parameter learns is directly proportional to the magnitude of its gradient.** If either the incoming activation (a_{in}) or the neuron error (δ_{out}) is very small, the resulting gradient is tiny. A tiny gradient means a microscopic weight update, causing the network to learn painfully slowly.

1.6.2 The Saturation Problem

Look at the formula for the output error (BP1):

$$\delta^L = \nabla_{a^L} C \odot \sigma'(z^L)$$

The gradient is multiplied by the derivative of the activation function (σ'). If you use a Sigmoid function, its values are squeezed between 0 and 1.

- If the neuron is highly active (output near 1) or highly inactive (output near 0), the curve becomes completely flat.
- A flat curve has a slope (derivative) of roughly 0.

When this happens, the neuron is said to be **saturated**. Because $\sigma'(z) \approx 0$, the entire δ term becomes 0. The error signal is *killed,* and the weights connected to that neuron stop updating. *Data Science Takeaway:* This is the exact reason why modern Deep Learning rarely uses Sigmoid for hidden layers, opting for ReLU instead.

1.6.3 The Vanishing Gradient Problem in Deep Networks

The saturation problem compounds geometrically as you build deeper networks.

According to BP2, the error is passed backwards layer by layer, multiplying by the activation derivative (σ') at each step. Because the derivative of a Sigmoid is always less than 0.25, you are repeatedly multiplying by fractions:

$$\delta \times 0.2 \times 0.1 \times 0.2 \ldots$$

By the time the error signal reaches the very first hidden layers of a deep network, the gradient has shrunk to microscopic levels (e.g., 10^{-15}). This is the **Vanishing Gradient Problem**. The early layers of the network, which are responsible for learning foundational features, receive almost no learning signal and fail to train.

1.6.4 The Complete Training Loop (Stochastic Gradient Descent)

backpropagation alone does not update the weights. It only computes the gradients. To actually learn, it must be combined with an optimization method. To train a model, we combine it with **Stochastic Gradient Descent (SGD)** using **Mini-Batches** and **Epochs**.

1. **Mini-Batch Selection:** Select a small subset of the training data (e.g., 32 images).
2. **Forward Pass:** Pass all 32 images through the network to calculate predictions.
3. **Calculate Error:** Compute the loss and apply BP1 for the output layer.
4. **Backpropagate:** Apply BP2 to push the error backwards through the hidden layers.
5. **Calculate Gradients:** Apply BP3 and BP4 to find the gradients for all weights and biases.
6. **Average and Update:** Average the 32 gradients together, multiply by the learning rate (η), and update the actual weights.
7. **Next Batch:** Move to the next batch until the entire dataset has been seen. This completes one **Epoch**.